LIVRE BLANC

REPRENEZ LE CONTRÔLE DE VOTRE BASE DE DONNÉES

ÉVALUEZ, CORRIGEZ ET STRUCTUREZ POUR PASSER À LA VITESSE SUPÉRIEURE.

DE THOMAS RICHARD

REPRENEZ LE CONTRÔLE DE VOTRE BASE DE DONNÉES

ÉVALUEZ, CORRIGEZ ET STRUCTUREZ POUR PASSER À LA VITESSE SUPÉRIEURE.

Vous avez créé votre base dans Airtable, SmartSuite ou Notion.
Elle fonctionne… à peu près.
Mais vous ressentez ces petits signaux :

- des infos en double,
- des champs qui ne servent plus,
- des automatisations capricieuses,
- des vues que personne ne comprend…

Ce livre blanc vous aide à **reprendre la main sur votre base** : modélisation, normalisation, gouvernance.
Le tout, avec des exemples concrets, des erreurs à éviter… et un plan d'action pour avancer.

INTRODUCTION

Aujourd'hui, de plus en plus d'entreprises — PME, startups, indépendants ou équipes projets — construisent leurs bases de données dans des outils modernes comme Airtable, SmartSuite, Notion ou encore Coda. Ces plateformes offrent une puissance et une flexibilité impressionnantes… à condition d'en maîtriser les fondations.

Mais en pratique, beaucoup de bases deviennent rapidement confuses, peu fiables, et difficiles à faire évoluer. Et c'est bien normal : modéliser proprement une base de données n'est pas inné, surtout sans formation technique.

👉 CE GUIDE EST LÀ POUR RÉPONDRE À UNE QUESTION ESSENTIELLE : « MA BASE DE DONNÉES EST-ELLE SAINE, FIABLE ET PRÊTE POUR LE FUTUR ? »

Ce livre blanc est conçu pour toutes celles et ceux qui :
- ont déjà construit une base sur Airtable, SmartSuite ou un outil similaire,
- se demandent si elle est bien structurée,
- ont rencontré des limites, de la confusion ou des bugs,
- veulent pouvoir s'appuyer sur une base solide, pérenne, évolutive,
- envisagent d'automatiser leurs processus et ont besoin d'une structure claire.

Que vous soyez dirigeant de PME, chef de projet, ops, consultant, ou freelance no-code, vous trouverez ici une méthode concrète pour évaluer et améliorer votre base.
Au fil de ce guide, vous allez :
- apprendre à **modéliser et structurer vos données**, en représentant clairement vos objets métier et les relations qui les unissent ;
- **normaliser vos données** pour en garantir la qualité, faciliter les recherches, les calculs et les automatisations ;
- et poser les bases d'une **vision claire et d'une gouvernance durable**, afin que votre base réponde aux besoins concrets de vos équipes tout en restant lisible et évolutive.

Objectif : vous donner une grille de lecture claire pour (re)prendre le contrôle de vos données.

1.
COMPRENDRE CE QU'EST UNE BASE DE DONNÉES RELATIONNELLE MODERNE

1.1 COMPRENDRE LES FONDAMENTAUX

🧠 L'OBJET MÉTIER : LE CŒUR DE VOTRE BASE

Tout part d'un bon découpage. Un "objet métier", c'est une chose **distincte**, **concrète** et **suivie dans le temps** dans votre activité : un client, un projet, une facture, un produit, un salarié…
Chacun de ces objets mérite sa propre **table** dans une base relationnelle. Pourquoi ? Parce qu'il peut exister de manière indépendante, et qu'il possède ses **propres caractéristiques** (nom, date, statut, montant, etc.).

📌 Par exemple :

- Un **client** a un nom, une entreprise, un e-mail.
- Un **projet** a un nom, une date de début, un statut.
- Une **facture** a un numéro, un montant, une date d'émission.

  👉 Ces objets ne sont pas interchangeables et doivent donc exister dans des tables séparées.

🧭 OBJET OU CARACTÉRISTIQUE ?

Une erreur fréquente consiste à confondre un **objet métier** avec une **caractéristique** d'un autre objet.

Parfois, la frontière est floue.
Voici quelques repères pour **distinguer un objet métier** d'un simple champ.
Un élément est un objet métier si :

- Il possède plusieurs attributs (nom, type, date de création…)
- Il peut exister indépendamment des autres objets

- Il est potentiellement **réutilisé ou lié** à d'autres objets (ex : un produit dans plusieurs factures)
- Vous avez besoin de **le filtrer, le suivre, l'exploiter** (ex : un consultant dans plusieurs projets)

Un élément est une caractéristique (champ) si :

- Il ne décrit qu'un aspect d'un objet plus grand
- Il n'a pas besoin d'exister seul
- Il n'est pas réutilisé ailleurs

🎓 Exemples pour s'entraîner à trancher :

- Le statut d'un projet : → un champ (caractéristique)
- Le type de produit vendu : → un champ… sauf si vous devez gérer une liste de types avec règles ou prix associés → dans ce cas, une table.
- Le contact principal d'un client : → souvent un champ (nom, téléphone) mais si vous avez plusieurs contacts ou des rôles associés → une table liée.

🎯 LA RÈGLE D'OR : UNE TABLE = UN OBJET MÉTIER

C'est ce qu'on appelle le principe de **"Single Source of Truth"** : chaque **donnée doit être modifiable à un seul endroit**, dans **la table de l'objet métier concerné**. Ensuite, vous pouvez l'afficher ailleurs grâce aux **liens** et aux **lookups** (recherches).

📌 Exemple :
Le nom d'un client ne devrait pas être saisi dans une table "Projets" ou "Factures". Il doit être modifié **uniquement dans la table "Clients"**, puis automatiquement affiché dans les autres tables via un champ lié ou une recherche.

Cette approche garantit que vos données sont **cohérentes et toujours à jour**, sans risque de contradiction ou de saisie manuelle inutile.

1.2 STRUCTURER LES RELATIONS

🔗 COMPRENDRE LES TYPES DE RELATIONS

Une base de données relationnelle repose… sur des **relations** ! Relier correctement vos objets métier entre eux est fondamental pour refléter la réalité de votre activité. On distingue trois grands types de liens :

1. La relation un-à-un (1–1)
Chaque élément d'une table est lié à un seul élément d'une autre table.

📌 Exemple : un **projet** = un **contrat**
Chaque projet a son propre contrat, et chaque contrat est associé à un seul projet.
✅ Dans ce cas, les deux tables sont reliées en 1–1.

2. La relation un-à-plusieurs (1–n)
Un élément d'une table est lié à **plusieurs** éléments dans une autre table.

📌 Exemple : un **client** = plusieurs **projets**
Un client peut avoir plusieurs projets, mais chaque projet n'a qu'un seul client.
✅ C'est le lien le plus courant : chaque projet contient un lien vers un client.

3. La relation plusieurs-à-plusieurs (n–n)
Plusieurs éléments d'une table peuvent être liés à plusieurs éléments dans une autre.

📌 Exemple : une **facture** contient plusieurs **produits**, et un **produit** peut apparaître dans plusieurs factures.
✅ Il faut alors créer une **table de liaison** entre les deux :

ex. une table "Lignes de facture" avec :

- un lien vers la facture
- un lien vers le produit
- et des champs spécifiques (quantité, prix unitaire…)

📐 BIEN MODÉLISER SES LIENS

Toujours partir de la réalité métier
Avant de créer un lien dans votre base, posez-vous une question simple : *comment ces éléments interagissent-ils dans la vraie vie ?*
Si un projet appartient à un seul client, alors le lien est de type un-à-plusieurs. Si un même produit peut apparaître dans plusieurs devis, c'est une relation plusieurs-à-plusieurs.
👉 La base doit refléter fidèlement le fonctionnement réel de votre entreprise.

Préférer les champs liés aux saisies manuelles
Un champ texte libre (comme "Nom du client") peut contenir des erreurs, des variations, ou des doublons. En utilisant un **champ lié** à une autre table, vous sécurisez vos données : pas d'orthographe approximative, pas de décalage entre l'étiquette affichée et la fiche client réelle.
👉 Cela garantit la cohérence et permet de retrouver et croiser les informations plus facilement.

Penser navigation et lecture croisée
Une bonne structure permet de naviguer naturellement d'un objet à l'autre. Par exemple, à partir d'un client, vous devez pouvoir visualiser tous ses projets, toutes ses factures, ou toutes ses commandes.
👉 C'est ce qu'on appelle une vision **multi-niveaux** ou **contextuelle** de vos données.

⚠️ ATTENTION AUX FAUSSES RELATIONS

✖ **Mauvaise pratique :**
Dans la table "Projets", vous ajoutez un champ texte "Nom du client" où vous tapez manuellement "Dupont SARL" à chaque nouveau projet.
→ Résultat : si le nom change ou si vous faites une faute de frappe, l'info est déconnectée de la réalité.
✅ **Bonne pratique :**
Créez un **lien vers la table "Clients"** dans la table "Projets". Cela vous permet de choisir le bon client dans une liste fiable. Ensuite, utilisez des **lookups** (recherches) pour faire remonter automatiquement les infos utiles (ex. email, statut du client, secteur…).
→ Résultat : vous évitez les doublons, les erreurs, et vous centralisez la donnée là où elle doit vivre.

1.3 BONNES PRATIQUES DE MODÉLISATION

Modéliser une base ne consiste pas seulement à créer des tables et des liens. C'est une démarche de clarification, d'anticipation et de maintenabilité. Voici quelques bonnes pratiques simples, mais fondamentales.

🏷️ NOMMEZ VOS TABLES ET VOS CHAMPS AVEC SOIN

Une base bien nommée est une base qu'on comprend sans documentation. Vos tables doivent refléter des objets concrets et vos champs des caractéristiques précises.
Évitez les abréviations obscures, les noms techniques ou les conventions issues d'Excel.

✅ Exemples de bons noms :
`Clients`, `Projets`, `Date de début`, `Type de contrat`, `Montant HT`

✖ À éviter :
`Tbl_01`, `BaseCL`, `Champ1`, `varNom`, `InfosDiverses`

Cela permet à n'importe qui (collègue, client, vous-même dans 6 mois) de comprendre ce que contient la base sans devoir tout ouvrir champ par champ.
C'est aussi **indispensable pour les automatisations**, où les noms de champs sont souvent utilisés tels quels (dans Make ou Zapier par exemple).

🧩 NE REGROUPEZ PAS TOUT DANS UNE SEULE TABLE

C'est l'erreur numéro 1 quand on débute : vouloir centraliser toutes les données dans une seule table, comme on le ferait dans un gros tableau Excel. Le résultat est souvent une table immense, avec des colonnes qui ne servent qu'à certaines lignes, et une confusion constante sur ce que représente chaque ligne.

✖ Exemple typique : une table "Suivi clients" qui contient les infos du client, ses projets, les dates de relance, les montants facturés, les retours des commerciaux…

✅ Solution saine :

- Une table "Clients" pour les données d'identité
- Une table "Projets" pour chaque mission réalisée
- Une table "Interactions" ou "Suivi" pour les échanges ou actions menées

Chaque table reste **spécialisée**, **épurée**, et **reliée** aux autres. On gagne en lisibilité, en qualité de données, et en agilité, comme vu précédemment. On se répète, mais c'est primordial !

> ***SMARTSUITE PERMET DE LIER ENTRE ELLE PLUSIEURS TABLE DU MÊME ESPACE DE TRAVAIL. C'EST UNE EXCELLENT FONCTIONNALITÉ POUR PLUS DE CLARTÉ DANS VOTRE ORGANISATION.***
> ***ON PEUT AVOIR PAR EXEMPLE UNE SEULE BASE CONTACTS QUI EST LIÉE AUX FACTURES, AUX ÉVÈNEMENTS ETC..***

📏 QUAND SÉPARER, QUAND FUSIONNER ?

La frontière n'est pas toujours évidente. Voici trois questions simples pour vous guider :

1. **Est-ce que ces deux éléments ont une vie propre ?**
   Par exemple, un projet vit indépendamment du client, et une facture vit indépendamment du projet auquel elle est rattachée.

2. **Est-ce qu'ils ont des attributs différents ?**
   Si vous avez des colonnes qui ne concernent qu'un type de ligne, c'est souvent un signe qu'il faut séparer.

3. **Est-ce qu'ils peuvent exister sans se référer directement l'un à l'autre ?**
   Par exemple, vous pouvez avoir une table "Ressources" avec des collaborateurs internes et des prestataires, qui ne sont pas liés à des projets au moment de leur création.

Quand ces conditions sont réunies, vous avez intérêt à **séparer**.

Mais attention : si deux éléments partagent strictement les mêmes caractéristiques et ne sont jamais utilisés séparément, une **fusion peut être plus pertinente** pour garder les choses simples.

🧠 PRIVILÉGIEZ LA COHÉRENCE À LA PERFECTION

Vouloir tout anticiper est tentant, mais souvent contre-productif. Une base parfaite n'existe pas, surtout dans un contexte en mouvement constant comme une PME. En revanche, une base **cohérente et bien documentée**, oui.

✅ Ce qu'il faut viser :

- Des noms clairs
- Une structure logique et stable
- Des liens bien nommés
- Un système compréhensible par d'autres que vous

N'hésitez pas à **documenter vos choix** (via une table dédiée, un champ "note interne", ou une vue "admin"). Et surtout : **faites des points réguliers** pour faire évoluer la structure en fonction des usages réels. C'est la clé pour éviter l'effet "usine à gaz".

1.4 ERREURS FRÉQUENTES

Même avec de bonnes intentions, certaines erreurs de modélisation reviennent très souvent. Les repérer permet de mieux comprendre ce qui fait la qualité d'une base relationnelle bien conçue.

❌ TOUT METTRE DANS UNE SEULE TABLE

C'est l'erreur la plus répandue chez les débutants : tout centraliser dans un seul tableau pour "voir tout au même endroit".
Résultat : une table avec des dizaines de colonnes, certaines inutilisées, et des lignes qui mélangent des natures d'information différentes.

🎯 Ce qu'il faut faire :
Identifier les **objets distincts** (clients, projets, tâches…) et leur créer une **table dédiée**. Puis relier les tables entre elles.

❌ DUPLIQUER LES DONNÉES D'UN OBJET MÉTIER

Renseigner manuellement le nom d'un client dans chaque table où il apparaît (projets, factures, tickets SAV…) peut sembler anodin. Mais cela crée une bombe à retardement : fautes de frappe, noms incohérents, ou données qui ne sont pas à jour partout.

🎯 Ce qu'il faut faire :
Une seule table "Clients", un seul endroit où modifier les données, et des **liens + lookups** pour les afficher ailleurs. C'est ça, le **Single Source of Truth**.

❌ LIER DES OBJETS SANS LOGIQUE FONCTIONNELLE

Parfois, on crée des liens "parce qu'on peut", sans que cela corresponde à une relation logique dans la réalité métier.
Par exemple : relier une table "Événements" à "Produits" parce qu'on a envie d'associer les deux, sans que cela soit justifié dans les processus réels.

🎯 Ce qu'il faut faire :
Toujours partir des **relations métier réelles**. Si deux objets ne sont pas naturellement liés dans votre activité, ne les reliez pas dans la base. Ou passez par une **table intermédiaire** avec un objectif clair.

❌ MULTIPLIER LES CHAMPS AU LIEU DE CRÉER DES LIGNES

Un grand classique : au lieu d'avoir une table "Rendez-vous" avec une ligne par RDV, on crée dans la table "Clients" les champs `RDV 1`, `RDV 2`, `RDV 3`, etc.

🎯 Ce qu'il faut faire :
Créez une **table liée "Rendez-vous"**, avec un champ "Date" et un lien vers le client. Vous pourrez ainsi gérer un nombre illimité de RDV, les trier, les filtrer, et les visualiser proprement.

❌ CRÉER DES TABLES "JUSTE POUR AFFICHER AUTREMENT"

Créer une deuxième table "Projets terminés" au lieu d'utiliser une **vue filtrée** est une erreur fréquente. Cela mène à des doublons et à une perte de contrôle.

🎯 Ce qu'il faut faire :
Utilisez les **vues** (filtrées, triées, regroupées…) pour afficher la même table selon différents critères, sans dupliquer vos données.

2.
NORMALISATION DES DONNÉES

2.1 COMPRENDRE LA NORMALISATION

✅ QU'EST-CE QU'UNE DONNÉE NORMALISÉE ?

Une donnée **normalisée**, c'est une donnée **propre**, **structurée**, et **interprétable** par votre base, vos formules, vos automatisations… et vos collègues.

Cela signifie qu'elle respecte un **format constant**, qu'elle est **stockée dans un champ adapté à sa nature**, et qu'elle peut être exploitée sans nettoyage préalable.

📌 Par exemple :

- Une **date normalisée** est saisie dans un champ de type "date" : vous pouvez ensuite trier, calculer des délais, afficher dans un calendrier.
- Un **montant normalisé** est un champ "nombre" : vous pouvez le sommer, comparer, ou déclencher une alerte si le total dépasse un seuil.
- Un **statut normalisé** est choisi parmi une liste prédéfinie : vous évitez les fautes de frappe et garantissez la cohérence des filtres.
-

À l'inverse, une donnée "non normalisée" est souvent libre, variable, et source d'erreurs.

🧹 POURQUOI C'EST ESSENTIEL ?

Tant que vous consultez votre base "à l'œil", les petites incohérences passent inaperçues. Mais dès que vous voulez **filtrer**, **calculer**, **regrouper**, ou **automatiser**, la qualité des données devient critique.

✖ Vous voulez filtrer "tous les projets signés"
→ mais certains ont "signé", d'autres "Signé", d'autres encore "ok client"…
👉 Vous passez à côté d'infos essentielles, ou vous devez corriger manuellement.

✖ Vous voulez calculer un total de factures
→ mais les montants sont au format texte, avec des virgules, des € ou des espaces.
👉 La formule ne fonctionne pas, ou vous obtenez un résultat erroné.

✖ Vous créez une automatisation qui envoie un mail quand un contrat est “validé”
→ sauf que dans certains cas, c’est écrit “Validé” avec une majuscule, ou “Ok signé”...
👉 L’automatisation ne se déclenche pas.

Bref : des données non normalisées **bloquent la puissance de votre base**.

⚙️ QUEL IMPACT SUR LES FILTRES, FORMULES, AUTOMATISATIONS ?

Quand vos données sont bien typées et bien formatées, tout devient plus simple. Vous pouvez :

- **Filtrer** rapidement par statut, catégorie, ou période
  (ex. : tous les devis “en attente” émis ce mois-ci)
- **Faire des calculs précis et robustes**
  (ex. : moyenne des montants facturés par type de projet)
- **Écrire des formules claires**
  (ex. : si statut = “Validé” ET montant > 5000 alors alerte = “Oui”)
- **Créer des automatisations fiables**
  (ex. : envoyer un rappel si un contrat n’est pas signé 7 jours après l’envoi)

> 🧠 À retenir : une base, c’est comme une machine. Elle ne sait travailler que sur des données claires, cohérentes, et bien rangées.

2.2 CHOISIR LE BON TYPE DE CHAMP

🧱 CHAQUE DONNÉE MÉRITE LE BON FORMAT

Le type de champ que vous choisissez a un impact direct sur la **qualité**, la **recherche**, le **tri**, la **fiabilité**, et même l'**automatisation** de votre base.

Voici quelques repères simples :

- 📆 Une **date** doit être un champ "date", pas du texte ("15 avril" ou "hier" sont trop ambigus).
- 💰 Un **montant** doit être un "nombre", sans € ni virgule dans le champ lui-même.
- 📌 Un **champ booléen** (oui/non) ? Utilisez une **case à cocher**, pas du texte ("oui", "ok", "validé").
- 🧾 Une **catégorie ou un statut** ? Utilisez un **champ "single select"** avec des valeurs bien définies.

Cela vous permet ensuite de trier, grouper, filtrer, et automatiser… sans effort.

🎨 SINGLE SELECT OU MULTI SELECT .. OU LIEN?

Un **single select** permet de choisir **une seule valeur** parmi une liste : idéal pour un statut (ex. : "Brouillon", "Envoyé", "Signé") ou un niveau de priorité.

Un **multi select** permet d'associer plusieurs étiquettes à un même élément : pratique pour des tags, des thématiques, des canaux de communication…

⚠️ Attention : les champs multiselect sont visuels et pratiques pour l'utilisateur, mais plus difficiles à exploiter dans les automatisations et formules, car chaque cellule contient une liste.

🔄 CRÉER UN CHAMP "SELECT" OU UNE VRAIE TABLE LIÉE ?

C'est l'une des questions les plus importantes quand on conçoit une base :
Dois-je créer un champ select… ou modéliser un nouvel objet métier ?

Voici des critères pour trancher :

Cas	Utiliser un *select*	Créer une table liée
La donnée est une **étiquette** simple, stable	✅ Oui (ex. statut, type, catégorie)	❌ Non
La donnée a des **caractéristiques propres** (nom, email, tarif...)	❌ Non	✅ Oui
Vous voulez pouvoir **filtrer ou regrouper rapidement**	✅ Oui	✅ Oui
Vous devez **lier plusieurs éléments** à un objet existant (relation 1–n ou n–n)*	❌ Non	✅ Oui
Vous avez besoin d'un **historique ou de stats détaillées** **	❌ Non	✅ Oui

🧾 Notes :

(*) Lier plusieurs éléments à un même objet
Un champ *select* ne permet pas de créer une **vraie relation** entre des enregistrements. Il affiche juste un mot ou une étiquette, sans "connexion".
Avec une table liée, vous pouvez :

- naviguer dans les deux sens (voir les projets d'un client, les produits d'un devis, etc.),
- utiliser des **lookups**, **rollups** ou **automatisations** sur les données associées.

Exemple :
Dans une table "Projets", un champ *select* "Client" ne permettra pas de retrouver facilement tous les projets d'un même client.
En revanche, un lien vers une table "Clients" permet d'afficher tous les projets rattachés à chaque fiche client.

(**) Historique et statistiques détaillées**

Un *select* ne garde aucune trace du passé. Impossible de savoir **quand** une valeur a été modifiée, ni combien de fois elle a été utilisée.
Une table liée permet de représenter **chaque événement comme une ligne** avec ses propres attributs (date, type, durée…), ce qui ouvre la voie à :

- des comptages, des moyennes, des graphes,
- une analyse dans le temps,
- ou des automatisations conditionnelles basées sur le volume ou la fréquence.

Exemple :
Pour suivre les interactions commerciales avec un prospect :

- Le champ multiselect "Échanges" donne une info visuelle mais inexploitable.
- Une table "Interactions" avec une date, un type, une durée permet un **véritable suivi dynamique**.

🛑 ÉVITEZ LES CHAMPS "FOURRE-TOUT"

Un champ “fourre-tout” est un champ qui essaie de contenir **plusieurs informations à la fois**, sans structure claire. Il devient vite un vide-poches numérique : on y met “ce qui ne rentre pas ailleurs”.

✖ Exemples typiques :

- “Commentaires / statut / remarques”
- “Infos diverses”
- “Autres éléments à noter”
- “Notes du commercial + retour client”

Le problème, c’est que ces champs :

- **ne sont pas filtrables** (vous ne pouvez pas extraire les lignes où le statut est “Urgent”)
- **ne sont pas exploitables** dans des formules ou des automatisations
- **créent de l’ambiguïté** sur qui peut les remplir, quand, et comment

Au bout de quelques semaines, le champ devient un fourre-tout **illisible**, que personne n’ose supprimer… mais que tout le monde ignore.

✅ Ce qu’il faut faire à la place

Identifiez les informations récurrentes dans ces champs, et créez des champs spécifiques :
Exemple : au lieu de “Commentaires / statut”, créez un champ “Statut” (select) + un champ “Commentaire interne” (texte long)

Nommez vos champs selon leur contenu, pas selon leur forme
Un champ “Notes” peut contenir n’importe quoi. Mais un champ “Retour client après démo” donne tout de suite une intention claire.

Si vraiment un champ est libre, précisez sa finalité dans sa description ou son nom :
ex. “Commentaire interne (non transmis au client)”

2.3 EXPLOITER LES DONNÉES : FORMULES, RECHERCHES, AUTOMATISATIONS

Normaliser ses données, ce n'est pas seulement "faire propre" : c'est rendre sa base **vivante et intelligente**. Une base bien structurée peut effectuer des **calculs**, **extraire des informations clés**, ou **déclencher automatiquement des actions**.
À l'inverse, une base mal formatée devient rigide, opaque, et source d'erreurs manuelles.

🧮 FORMULES : AUTOMATISER LA LOGIQUE MÉTIER

Les champs calculés permettent d'ajouter de l'**intelligence métier** dans votre base : délais, conditions, indicateurs, statuts dynamiques, scores, titres générés…

📌 Exemples d'usage :

- **Calcul de durée** entre deux dates : Date de fin - Date de début
- **État d'un projet** : SI(Date aujourd'hui > Date limite ; "En retard" ; "Dans les temps")
- Création de titres automatiques : `Nom du client & " — " & Mois`
- **Score d'opportunité** : pondérer des critères et sortir un score global (ex. budget, priorité, intérêt)

Ces formules sont puissantes **à une condition** : que les champs utilisés soient bien typés.
❌ Si une date est enregistrée en texte ("15 avril 2024"), vous ne pouvez pas calculer de délai.
❌ Si un montant est saisi comme "3 000 €", il ne sera pas reconnu comme un nombre.
👉 Une donnée mal normalisée rend vos formules inutilisables… ou fausses.

🔍 LOOKUPS ET ROLLUPS : FAIRE CIRCULER L'INFORMATION ENTRE TABLES

Dans une base relationnelle, chaque table contient un objet métier. Mais souvent, vous avez besoin de **voir ou exploiter une info d'une autre table**. C'est là qu'interviennent deux outils clés :

- Le **lookup** (ou "recherche") : il permet d'afficher un champ provenant d'un lien.
  Ex. : afficher l'adresse email du client dans la table "Projets".

- Le **rollup** : il permet de faire un **calcul sur tous les enregistrements liés**.
  Ex. : calculer la somme des factures associées à un client.

📌 Autres exemples d'usage :

- Calculer le montant total facturé par client
- Afficher la **dernière date de relance** dans une fiche prospect
- Compter le nombre de tâches en retard dans un projet

Les lookups et rollups évitent de dupliquer l'information, tout en rendant la base plus lisible, connectée et actionnable.

🤖 AUTOMATISATIONS : DÉCLENCHER DES ACTIONS SANS INTERVENTION HUMAINE

Une fois les données bien normalisées, vous pouvez automatiser une grande variété d'actions :

📌 Exemples :

- Envoyer un mail au client si le **statut du projet passe à "Validé"**
- Créer une tâche dans ClickUp quand une **opportunité dépasse 10 000 €**
- Générer un PDF dès qu'une facture est marquée "À envoyer"
- Publier une alerte Slack quand un **contrat est signé**

Ces automatisations fonctionnent **sur des conditions claires**. Mais pour ça, il faut que les données soient **standardisées et prévisibles**.

❌ Si chacun écrit "Validé", "valide", "ok client", "Go"... les outils d'automatisation ne savent pas reconnaître la condition.

✅ Si vous utilisez un *select*, l'automate sait exactement quand agir.

👉 La qualité de vos automatisations dépend directement de la **rigueur de vos données**.

📉 CE QUE VOUS NE POUVEZ PAS FAIRE AVEC DES DONNÉES MAL NORMALISÉES

Les limites apparaissent vite quand la structure de vos champs est trop libre :

Ce que vous voulez faire	Pourquoi ça ne marche pas
Calculer un total de factures	Les montants sont saisis en texte, ou contiennent des
Trier des projets par date	Les dates sont tapées en texte libre ("mi-avril", "hier",
Créer une formule conditionnelle	Les statuts sont écrits de manière incohérente
Filtrer toutes les opportunités "chaudes"	Les valeurs ne sont pas normalisées ou sont mal typées
Regrouper les projets par client	Les clients sont saisis à la main (pas liés), donc non regroupables

Réutiliser un contact ou produit dans plusieurs tables	Ce n’est pas un objet métier lié, juste un nom en texte

2.4 ERREURS FRÉQUENTES

Même avec une base bien modélisée, beaucoup d'équipes tombent dans les pièges de la **non-normalisation des données**. Ces erreurs sont souvent subtiles au départ, mais elles ont un effet domino sur les formules, les automatisations, les filtres… et donc sur la fiabilité de la base.
Voici les pièges les plus fréquents à repérer et corriger.

❌ Tout laisser en texte libre

Par défaut, la plupart des utilisateurs saisissent les données **sous forme de texte** : pratique à court terme, mais désastreux à moyen terme.

✖ Problème :

- "Oui", "OUI", "ok", "validé", "ok client" = 5 manières de dire la même chose
- "15 avril", "15/04/24", "lundi prochain" = impossible à trier ou à filtrer
- "3 000 €", "3000€", "3000" = impossible à sommer sans nettoyage

✅ Ce qu'il faut faire :

Utiliser des champs **typés** : date, nombre, case à cocher, select…
Cela garantit l'unicité des formats et rend les données **exploitables automatiquement**.

❌ Utiliser un seul champ pour plusieurs informations

Un autre piège courant : mélanger plusieurs infos dans une seule cellule.

✖ Exemple :

Un champ "Résumé appel client" contient :
"Appelé le 12/04 – OK pour avancer – Besoin d'un devis d'ici vendredi – Statut : chaud"
Dans ce cas :

- Vous ne pouvez pas filtrer les appels en attente
- Vous ne pouvez pas savoir qui a été relancé
- Vous ne pouvez pas automatiser un rappel ou générer une tâche

✅ Ce qu'il faut faire :

Découper l'information :

- Champ "Date de contact" (type date)
- Champ "Statut" (select)
- Champ "Commentaire libre" (texte long, facultatif)

👉 Une cellule = une information = un usage clair.

❌ Laisser l'utilisateur saisir librement des catégories, types ou statuts

C’est une erreur très fréquente lorsqu'on veut “laisser de la souplesse”.
Mais en réalité, cela **désorganise totalement la base**.

✖ Exemple :

Champ “Statut” en texte libre : certains mettent “OK”, d’autres “En cours”, d’autres “En cours de traitement”, ou “à faire”...

Conséquences :

- Les filtres ne fonctionnent pas
- Les regroupements sont confus
- Les automatisations ne se déclenchent pas
- Vous n’avez aucune vue fiable

✅ Ce qu’il faut faire :

Créer un champ *single select* avec une liste fermée de statuts.
Cela force la **cohérence**, tout en permettant une bonne lisibilité et un pilotage plus précis.

👉 Et si vous avez besoin de plus de détails, vous pouvez toujours ajouter un champ “commentaire”.

❌ Sous-estimer l’impact du mauvais format sur les automatisations

Les outils d’automatisation comme Make, Zapier, Airtable Automations… sont très puissants **mais très stricts**.
Ils attendent des données **propres et prévisibles**.

✖ Si un statut peut être écrit de 5 manières différentes
→ l’automatisation ne se déclenchera pas

✖ Si un montant est saisi comme “5000€”
→ le filtre “si montant > 4000” ne fonctionnera pas

✖ Si une date est vide ou mal typée
→ le calcul “+ 7 jours” renverra une erreur

✅ Ce qu’il faut faire :

- Uniformiser les statuts avec des *select*
- Séparer les unités (montant en nombre, devise dans un autre champ si besoin)
- Utiliser systématiquement les **types de champs natifs** proposés par l’outil

👉 Une automatisation fiable dépend d’une donnée bien formatée. C’est une règle absolue.

Souhaites-tu que je compile l’ensemble du **chapitre 2 complet** pour relecture et ajustements, ou que l’on enchaîne directement avec le **chapitre 3 – Vision et gouvernance** ?

3.
VISION ET GOUVERNANCE

3.1 CONCEVOIR UNE BASE ORIENTÉE USAGE

Une base bien modélisée et bien normalisée, c'est une bonne fondation. Mais pour qu'elle soit **vraiment utile**, encore faut-il qu'elle soit **conçue pour les gens qui l'utilisent**.
C'est toute la différence entre une base "propre" et une base **vivante, compréhensible et actionnable au quotidien**.

🎯 PENSER À L'UTILISATEUR FINAL

Votre base n'est pas qu'un entrepôt de données. C'est un outil que des humains vont consulter, modifier, utiliser pour décider ou pour agir.

Elle doit donc être structurée en fonction de **leurs besoins concrets** : retrouver l'info rapidement, voir ce qu'ils doivent faire, savoir ce qui est prioritaire, comprendre un historique, etc.

📌 Exemples de profils utilisateurs :

- un commercial qui veut voir **ses opportunités actives**
- un chef de projet qui suit **les livrables en attente**
- un dirigeant qui consulte les données consolidées en lecture seule
- un assistant qui saisit **les nouveaux prospects**

👉 Chaque utilisateur a besoin d'un **accès ciblé**, avec une interface adaptée à son rôle.

👁️ TRAVAILLER AVEC DES VUES CLAIRES ET FILTRÉES

Au lieu d'imposer à tous une vue unique, on peut créer des **vues personnalisées** : filtres, tris, regroupements, colonnes visibles…
Cela permet de présenter les mêmes données de manière **contextuelle**, selon les usages.

✅ Exemple : une table "Projets" peut avoir :

- une vue "En cours" (projets actifs, triés par date de fin)

- une vue “À facturer” (filtrée sur statut)
- une vue “Archivés” (en lecture seule)

Nous reviendrons en détail sur le pouvoir des vues dans la section suivante.

👷 ADAPTER L’INTERFACE AUX RÔLES ET À L’USAGE

Toutes les personnes qui consultent ou manipulent une base n’ont pas les mêmes besoins, ni les mêmes responsabilités.
Adapter l’interface, ce n’est pas un luxe : c’est **une nécessité pour maintenir l’adhésion, limiter les erreurs, et fluidifier l’expérience**. Cela permet également de limiter l’accès à des informations sensibles, comme ça peut être le cas dans des domaines RH, finances etc..

> 📌 Par exemple :
> - Un utilisateur opérationnel a besoin d’un formulaire simple pour **ajouter des données** (ex. un formulaire de création de prospect)
> - Un manager veut consulter un **suivi synthétique**, sans champs techniques ni champs modifiables
> - Un collaborateur externe doit pouvoir **saisir des infos**, mais sans voir les données stratégiques ou internes

Dans des outils comme **Airtable**, la fonctionnalité **“Interface”** permet de créer des interfaces visuelles distinctes pour chaque rôle ou besoin :

- tableaux de bord par équipe,
- fiches épurées pour les contributeurs,
- filtres prédéfinis,
- champs masqués ou verrouillés à la modification.

Cette **couche front** transforme une base “brute” en un outil métier **sur mesure**, plus clair et plus rassurant.
🎯 Une base bien structurée mais mal présentée reste inutilisable pour ceux qui ne la comprennent pas.

🧘 UNE BASE BIEN CONÇUE EST UNE BASE RASSURANTE

Un bon modèle de données, c’est la fondation.
Mais une **expérience de consultation fluide, claire, contextualisée**, c’est ce qui permet à la base d’être adoptée et utilisée au quotidien.

Quand chacun a **une vue adaptée à ses tâches**, une interface pensée pour lui, et un périmètre d'action bien défini :

- les utilisateurs sont **autonomes**, car ils savent où agir
- les erreurs sont **limitées**, car ils ne risquent pas de toucher à ce qu'ils ne doivent pas
- les échanges deviennent **plus fluides**, car chacun partage une représentation commune
- la base devient **un outil d'équipe**, et non un fardeau technique réservé à un expert

📌 Une bonne gouvernance commence par une bonne lisibilité.
Quand la base devient un espace partagé, lisible, et aligné sur les besoins concrets, elle devient un **levier de productivité** – et non un frein.

3.2 UTILISER LES VUES POUR STRUCTURER L'EXPÉRIENCE

Les **vues** sont l'un des outils les plus puissants – et les plus sous-estimés – dans une base relationnelle moderne.
Elles permettent d'afficher **les mêmes données**, mais selon **des angles adaptés à chaque usage, chaque rôle, chaque contexte**.
Une base bien structurée mais sans vues bien pensées… reste une base difficile à exploiter.

🧭 UNE VUE, CE N'EST PAS "JUSTE UN FILTRE"

On confond souvent les vues avec un simple tri ou un affichage personnalisé. En réalité, une **vue est une représentation contextuelle d'un objet métier**, qui rend l'information lisible et actionnable.

📌 Exemple : une table "Tâches" peut avoir :

- une vue "À faire cette semaine" (filtrée sur échéance proche + statut non terminé)
- une vue "Par collaborateur" (groupée par responsable)
- une vue "Tâches terminées" (en lecture seule, archivées automatiquement)

👉 Cela permet à chacun de **retrouver rapidement ce qui le concerne**, sans être noyé dans la masse d'informations.

🧰 DES VUES POUR CHAQUE RÔLE, CHAQUE MOMENT

En combinant **filtres**, **tris**, **groupes** et **colonnes visibles**, vous pouvez créer des vues adaptées à :

Usage	Exemple de vue
👷 Exécution	“Mes tâches à faire aujourd’hui”
🧠 Suivi stratégique	“Projets en retard – par équipe”
📤 Préparation	“Factures à émettre cette semaine”
🧾 Contrôle qualité	“Fiches clients incomplètes”
🗃️ Archivage	“Commandes terminées – triées par mois”

Chaque vue est comme une **fenêtre métier** sur votre base.

💡 PENSER “VUES” AVANT DE CRÉER DE NOUVELLES TABLES

L’une des erreurs fréquentes est de **multiplier les tables** à la place de créer des vues.

❌ Mauvaise pratique :
créer une table “Projets terminés”, une autre “Projets en cours”, une autre “Projets à venir”
✅ Bonne pratique :
créer une seule table “Projets”, et des **vues filtrées** selon le statut.
👉 Cela évite les doublons, garde une structure propre, et simplifie les automatisations.

⚙️ UTILISER LES VUES COMME DÉCLENCHEURS D’AUTOMATISATIONS

Les vues peuvent aussi servir de **déclencheur logique pour des automatisations**.

Dans Airtable, par exemple, vous pouvez configurer un scénario qui s’active **dès qu’un enregistrement entre dans une certaine vue** (c’est-à-dire, quand il remplit un ensemble de conditions métier).

📌 Exemple :

- Une facture entre dans la vue “À envoyer” → déclenchement d’un email automatique
- Une tâche entre dans “En retard + non assignée” → création d’une alerte Slack
- Un prospect passe en “À recontacter” → ajout à une séquence de relance

👉 L'avantage de cette approche, c'est que **les règles métiers sont visuelles et modifiables dans la base elle-même**, sans avoir à plonger dans un scénario complexe dans Make ou Zapier.
On gagne en **transparence**, en **agilité**, et on permet à des non-techniciens de gérer eux-mêmes la logique.

📚 À PROPOS DES TYPES DE VUES

Les outils comme Airtable ou SmartSuite proposent différents types de vues :
table, kanban, calendrier, galerie, timeline, grille hiérarchique, etc.
Chacun a ses avantages, selon le type d'objet et le besoin métier.
🎯 Une tâche a tout intérêt à être vue en **kanban** par statut,
une planification projet en **timeline**,
un suivi événementiel en **calendrier**,
un catalogue produit en **galerie**.
Nous ne rentrerons pas ici dans le détail de chaque type de vue – cela fera l'objet d'un **futur livre blanc dédié à l'optimisation des vues**.

3.3 GÉRER LES DROITS ET ÉVITER LE CHAOS

Plus une base est utilisée par différentes personnes, plus elle devient **fragile** si les accès ne sont pas maîtrisés. Sans un minimum de gouvernance, elle peut vite se transformer en champ de mines : champs supprimés, formules cassées, données écrasées par erreur…

Mettre en place une bonne gestion des droits, ce n'est pas fliquer : c'est **protéger la base**, **fluidifier la collaboration** et **responsabiliser les utilisateurs**.

🚫 TOUT LE MONDE N'A PAS BESOIN DE TOUT MODIFIER

Par défaut, dans de nombreux outils (notamment Airtable), tous les utilisateurs ont les mêmes droits… sauf si vous les définissez précisément.

Cela peut mener à des situations risquées :

- des utilisateurs qui modifient des champs techniques sans s'en rendre compte
- des automatismes qui ne fonctionnent plus parce que quelqu'un a changé un nom de champ
- des filtres qui disparaissent
- ou pire : des données supprimées par accident

🎯 Règle simple : seuls les rôles concernés doivent pouvoir modifier une donnée.

🔒 CRÉER DES ZONES SÛRES : VUES EN LECTURE SEULE, CHAMPS VERROUILLÉS

Quand votre outil le permet, vous pouvez :

- restreindre les droits par rôle ou par utilisateur
- créer des vues en lecture seule
- cacher ou verrouiller certains champs
- empêcher la suppression d'enregistrements

📌 Exemple :

- Une vue "Suivi projets – management" consultable en lecture seule
- Une interface de saisie où seuls les champs nécessaires sont visibles
- Un champ "Statut automatisé" protégé contre l'édition manuelle

Ces protections sont particulièrement importantes dans des bases partagées avec des partenaires externes, des clients ou des équipes non techniques.

👨‍🏫 MIEUX VAUT GUIDER QUE BRIDER

L'objectif n'est pas de tout verrouiller, mais de **rendre l'expérience plus claire et plus fluide**.

Un utilisateur qui sait exactement ce qu'il peut faire — et ce qu'il ne risque pas de casser — travaille **plus vite, avec plus de confiance**.

Et si vous travaillez avec des outils comme **Airtable Interfaces**, vous pouvez même aller plus loin :

- ne montrer que certaines vues selon le rôle
- proposer des formulaires de saisie guidés
- masquer la structure de la base à ceux qui n'en ont pas besoin

✅ Cela réduit les erreurs, limite les incompréhensions, et améliore l'adoption.

3.4 PILOTER L'ÉVOLUTION DE LA BASE

Créer une base efficace, ce n'est pas un projet qu'on coche une fois pour toutes. C'est un **système vivant**, qui doit s'adapter aux besoins, aux équipes, aux outils, aux règles du métier.
Sans vision d'ensemble ni entretien régulier, une base bien pensée peut facilement devenir un **patchwork illisible**, où chaque amélioration masque une couche de bricolage précédente.

🧭 PARTAGER UNE VISION CLAIRE

La première condition pour faire évoluer une base intelligemment, c'est de **savoir pourquoi elle existe**, à quoi elle sert, et pour qui.

> 📌 Questions à se poser :
> - Qui utilise cette base, et pour faire quoi ?
> - Quels sont les objectifs métiers principaux ?
> - Quelles sont les infos vraiment critiques à suivre ?
> - Quels usages sont prioritaires, et lesquels sont accessoires ?

Ce cadre permet de **filtrer les demandes d'ajout ou de modification** : tout ce qui est ajouté sans réflexion globale crée une dette fonctionnelle.

📘 DOCUMENTER LA STRUCTURE ET LES CONVENTIONS

Une base qui évolue sans documentation finit par devenir incompréhensible — même pour son créateur.

> 🎯 Bonnes pratiques :
> - Ajouter des descriptions aux tables, champs et vues
> - Créer une **vue ou une table dédiée à la documentation** (règles métier, conventions de nommage, historiques de modif…)
> - Inclure des **notes internes** pour expliquer la logique de certaines formules ou automatisations
> - Conserver un log de modifications structurelles

Même une base simple mérite un minimum de traces pour garantir sa maintenabilité.

🧹 ÉVITER L'EFFET "PATCHWORK" : REPENSER RÉGULIÈREMENT

Au fil du temps, les besoins changent, de nouveaux cas apparaissent, des automatisations s'ajoutent…
Sans garde-fou, on finit par empiler des exceptions, contourner les limites, créer des liens de secours ou ajouter des champs "bidouilles".

✖ Symptômes du patchwork :

- Des champs qui ne servent plus mais qu'on n'ose pas supprimer
- Des vues jamais utilisées
- Des automatisations devenues instables
- Des noms de champs ou tables incompréhensibles

✅ Ce qu'il faut faire :

- Prendre un temps de revue structurelle tous les 3 à 6 mois
- Réinterroger les tables et les relations
- Supprimer ce qui ne sert plus
- Refactorer les anciens éléments en fonction des nouveaux besoins

👉 C'est exactement comme du code ou un process d'équipe : ça s'entretient, ou ça dérive.

🚀 Gouvernance = longévité

Une base bien conçue vous fait gagner du temps, sécurise vos données, structure vos process. Mais seule une base bien gouvernée garde cette valeur dans le temps.

📌 **En résumé :**

- Ayez une vision claire de ce que la base doit servir
- Structurez vos rôles, vues, droits et interfaces
- Mettez en place une discipline d'entretien
- Et surtout : n'ajoutez jamais dans l'urgence ce que vous n'êtes pas prêt à maintenir demain

🎯 CONCLUSION – VERS UNE BASE PLUS FIABLE, PLUS UTILE, PLUS DURABLE

Vous l'avez vu à travers ce guide : une base de données relationnelle ne se résume pas à quelques tables et des colonnes bien nommées.
C'est un **système vivant**, au service de vos métiers, de vos processus, et de vos décisions.

📐 MODÉLISATION

Une base saine commence par une **bonne structure** :
1 table = 1 objet métier, des relations claires, des liens logiques.
On évite les doublons, on clarifie les liens, on représente fidèlement la réalité de votre activité.

🧹 NORMALISATION

Des données bien formatées sont **exploitables, fiables, actionnables**.
Vous pouvez chercher, calculer, automatiser sans bricoler.
C'est la différence entre une base qu'on subit, et une base qui travaille pour vous.

🧭 VISION & GOUVERNANCE

Enfin, une base doit être **compréhensible, évolutive, et bien pilotée**.
Vues, droits, documentation, rôles : ce sont ces éléments qui garantissent la durabilité de votre base, même quand les équipes, les outils ou les objectifs évoluent.

🖐️ ET MAINTENANT ?

Ce livre blanc ne vous donne pas toutes les réponses. Mais il vous offre un cadre clair pour :

- **auditer** votre base actuelle
- identifier vos points faibles
- restructurer proprement
- ou poser des fondations solides pour une nouvelle base

Prenez le temps de revoir vos tables, vos champs, vos liens. Même quelques ajustements bien ciblés peuvent transformer votre quotidien.

🛠️ UN OUTIL BONUS POUR ALLER PLUS LOIN

Pour compléter ce guide, vous pouvez télécharger :

- une **checklist de conception** pour créer une base saine dès le départ
  ou
- un **quiz d'auto-évaluation** pour tester la fiabilité de votre base actuelle

(Ce contenu est gratuit et accessible via le lien fourni.)

🤝 BESOIN D'UN REGARD EXTÉRIEUR ?

Vous préférez être accompagné dans cette démarche ?
Je vous aide à faire le point, restructurer, documenter, ou automatiser vos bases avec clarté et méthode.
→ **Contactez-moi pour un diagnostic personnalisé**
→ Ou abonnez-vous à la newsletter pour recevoir des conseils sur Airtable, SmartSuite et l'automatisation.

TESTEZ VOTRE BASE DE DONNEES

Ce quizz de 20 questions vous permettra d'avoir une première idée du niveau de conception de votre base

https://forms.fillout.com/t/8QcCsq4MZGus



À PROPOS DE L'AUTEUR

Thomas Richard accompagne les PME, agences et indépendants dans la structuration et l'automatisation de leurs outils métier.
Spécialiste des bases de données no-code (Airtable, SmartSuite, Notion…), il conçoit des systèmes clairs, fiables et évolutifs, au service de l'efficacité des équipes.

👉 En savoir plus : www.thomrichard.com

👉 Prendre contact : rendez vous découverte

👉 Newsletter « Work smart »